

Core Language Working Group "ready" Issues for the February, 2022 meeting

References in this document reflect the section and paragraph numbering of document [WG21 N4901](#).

1726. Declarator operators and conversion function

Section: 11.4.8.3 [class.conv.fct] **Status:** ready **Submitter:** James Widman **Date:** 2013-08-02

Presumably the following example is intended to be ill-formed:

```
struct A {  
    (*operator int*());  
};  
A a;  
int *x = a; // Ok?
```

It is not clear, however, which rule is supposed to reject such a *member-declaration*.

Proposed resolution (December, 2021):

Change 11.4.8.3 [class.conv.fct] paragraph 1 as follows, splitting the paragraph as indicated:

A member function of a class x with a name of the form

conversion-function-id:
 operator conversion-type-id

conversion-type-id:
 type-specifier-seq conversion-declarator_{opt}

conversion-declarator:
 ptr-operator conversion-declarator_{opt}

A declaration whose *declarator-id* has an *unqualified-id* that is a *conversion-function-id* declares a *conversion function*; its *declarator* shall be a function declarator (9.3.4.6 [dcl.fct]) of the form

***ptr-declarator* (*parameter-declaration-clause*) *cv-qualifier-seq_{opt}*
 *ref-qualifier-seq_{opt} noexcept-specifier_{opt} attribute-specifier-seq_{opt}***

where the *ptr-declarator* consists solely of an *id-expression*, an optional *attribute-specifier-seq*, and optional surrounding parentheses, and the *id-expression* has one of the following forms:

- in a *member-declaration* that belongs to the *member-specification* of a class or class template but is not a friend declaration (11.8.4 [class.friend]), the *id-expression* is a *conversion-function-id*;
- otherwise, the *id-expression* is a *qualified-id* whose *unqualified-id* is a *conversion-function-id*.

A **conversion function** shall have no parameters and **shall be a non-static member function of a class or class template *x*; it** specifies a conversion from *x* to the type specified by the *conversion-type-id*, interpreted as a *type-id* (9.3.2 [dcl.name]). ~~Such functions are called conversion functions.~~

A *decl-specifier* in the *decl-specifier-seq* of a conversion function (if any) shall **not** be ~~neither~~ a *defining-type-specifier* ~~nor static~~. The type of the conversion function (9.3.4.6 [dcl.fct]) is “**noexcept**_{opt} function taking no parameter **cv-qualifier-seq**_{opt} **ref-qualifier**_{opt} returning *conversion-type-id*”.

A conversion function is never used to convert a (possibly cv-qualified) object to the (possibly cv-qualified) same object type (or a reference to it), to a (possibly cv-qualified) base class of that type (or a reference to it), or to cv void.¹⁰² [Example 1:...

2494. Multiple definitions of non-odr-used entities

Section: 6.3 [basic.def.odr] **Status:** ready **Submitter:** Hubert Tong **Date:** 2021-07-02

According to 6.3 [basic.def.odr] paragraph 10,

Every program shall contain exactly one definition of every non-inline function or variable that is odr-used in that program outside of a discarded statement (8.5.2 [stmt.if]); no diagnostic required.

This wording could be interpreted as allowing multiple definitions of non-inline variables and functions if they are not odr-used. That is presumably not the intent.

Notes from the August, 2021 teleconference:

CWG observed that there is a similar problem in paragraph 13. See also issue 1849.

Proposed resolution, December, 2021:

1. Change 6.3 [basic.def.odr] paragraph 1 as follows:

Each of the following is termed a *definable item*:

- a class type (Clause Clause 11 [class]),
- an enumeration type (9.7.1 [dcl.enum]),
- a function (9.3.4.6 [dcl.fct]),
- a variable (6.1 [basic.pre]),

- **a templated entity (13.1 [temp.pre]),**
- **a default argument for a parameter (for a function in a given scope) (9.3.4.7 [dcl.fct.default]), or**
- **a default template argument (13.2 [temp.param]).**

No translation unit shall contain more than one definition of any ~~variable, function, class type, enumeration type, template, default argument for a parameter (for a function in a given scope), or default template argument~~ **definable item**.

2. Change 6.3 [basic.def.odr] paragraph 10 as follows:

Every program shall contain ~~exactly~~ **at least** one definition of every ~~non-inline~~ function or variable that is odr-used in that program outside of a discarded statement (8.5.2 [stmt.if]); no diagnostic required. The definition...

3. Change 6.3 [basic.def.odr] paragraph 13 as follows:

~~There can be more than one definition of a~~

- ~~class type (Clause Clause 11 [class]),~~
- ~~enumeration type (9.7.1 [dcl.enum]),~~
- ~~inline function or variable (9.2.8 [dcl.inline]),~~
- ~~templated entity (13.1 [temp.pre]),~~
- ~~default argument for a parameter (for a function in a given scope) (9.3.4.7 [dcl.fct.default]), or~~
- ~~default template argument (13.2 [temp.param])~~

~~in a program provided that each definition appears in a different translation unit and the definitions satisfy the following requirements.~~ **For any definable item D with definitions in multiple translation units,**

- **if D is a non-inline non-templated function or variable, or**
- **if the definitions in different translation units do not satisfy the following requirements,**

the program is ill-formed; a diagnostic is required only if the definable item is attached to a named module and a prior definition is reachable at the point where a later definition occurs. Given such an ~~entity D defined in more than one translation unit~~ **item**, for all definitions of D, or, if D is an unnamed enumeration, for all definitions of D that are reachable at any given program point, the following requirements shall be satisfied...

4. Delete 6.3 [basic.def.odr] paragraph 15:

~~If these definitions do not satisfy these requirements, then the program is ill-formed; a diagnostic is required only if the entity is attached to a named module and a prior definition is reachable at the point where a later definition occurs.~~

2499. Inconsistency in definition of pointer-interconvertibility

Section: 6.8.3 [basic.compound] **Status:** ready **Submitter:** Jason Merrill **Date:** 2021-07-29

The changes for issue 2254 included the following:

Change 6.8.3 [basic.compound] bullet 4.3 as follows:

Two objects *a* and *b* are *pointer-interconvertible* if:

- ...
- one is a standard-layout class object and the other is the first non-static data member of that object, or, if the object has no non-static data members, ~~the first~~ **any** base class subobject of that object (11.4 [class.mem]), or

This should also have removed the phrase,

or, if the object has no non-static data members,

since the change to 11.4 [class.mem] paragraph 25 specifies that all bases of a standard-layout class have the same address, regardless of whether the derived class has non-static data members.

Proposed resolution (November, 2021):

Change 6.8.3 [basic.compound] bullet 4.3 as follows:

Two objects *a* and *b* are *pointer-interconvertible* if:

- ...
- one is a standard-layout class object and the other is the first non-static data member of that object; ~~or, if the object has no non-static data members,~~ any base class subobject of that object (11.4 [class.mem]), or
- ...

2502. Unintended declaration conflicts in nested statement scopes

Section: 6.4.3 [basic.scope.block] **Status:** ready **Submitter:** Jens Maurer **Date:** 2021-08-26

The changes of P1787R6 inadvertently made constructs like

```
if (int a = 1)
  if (int a = 1)
    ...
```

ill-formed.

Proposed resolution (September, 2021):

Change 6.4.3 [basic.scope.block] bullet 2.2 as follows:

If a declaration whose target scope is the block scope S of a

- *compound-statement* of a *lambda-expression*, *function-body*, or *function-try-block*,
- substatement of a selection or iteration statement **that is not itself a selection or iteration statement**, or
- *handler* of a *function-try-block*

potentially conflicts with a declaration whose target scope is the parent scope of S , the program is ill-formed.

(See [editorial issue 4843](#).)

2506. Structured bindings and array cv-qualifiers

Section: 9.6 [dcl.struct.bind] **Status:** ready **Submitter:** Barry Revzin **Date:** 2018-12-11

According to 9.6 [dcl.struct.bind] paragraph 1,

A structured binding declaration introduces the identifiers $v_0, v_1, v_2, \dots$ of the *identifier-list* as names of *structured bindings*. Let *cv* denote the *cv-qualifiers* in the *decl-specifier-seq* and S consist of the *storage-class-specifiers* of the *decl-specifier-seq* (if any). A *cv* that includes *volatile* is deprecated; see D.5 [depr.volatile.type]. First, a variable with a unique name e is introduced. If the *assignment-expression* in the initializer has array type A and no *ref-qualifier* is present, e is defined by

$$\text{attribute-specifier-seq}_{\text{opt}} S \text{ cv } A \text{ } e \text{ } ;$$

and each element is copy-initialized or direct-initialized from the corresponding element of the *assignment-expression* as specified by the form of the initializer.

This means that in an example like

```
const int arr[1]{};
auto [i] = arr;
```

i is a reference to `const int`. Presumably the fact that the array is copied should drop the array's cv-qualification.

Proposed resolution (December, 2021):

Change 9.6 [dcl.struct.bind] paragraph 1 as follows:

If the *assignment-expression* in the initializer has array type **cv1** A and no *ref-qualifier* is present, e is defined by...

2509. *decl-specifier-seq* in *lambda-specifiers*

Section: 7.5.5.1 [expr.prim.lambda.general] **Status:** ready **Submitter:** Jens Maurer **Date:** 2021-10-28

(From editorial issue [2338](#).)

Use of *decl-specifier-seq* in the production for *lambda-specifiers* is too general and should be restricted.

Proposed resolution (December, 2021):

1. Change the grammar in 7.5.5.1 [expr.prim.lambda.general] as follows:

```
lambda-specifiers:
    decllambda-specifier-seqopt noexcept-specifieropt attribute-specifier-seqopt trailing-return-
    typeopt
lambda-specifier:
    consteval
    constexpr
    mutable
lambda-specifier-seq:
    lambda-specifier
    lambda-specifier lambda-specifier-seq
```

2. Change 7.5.5.1 [expr.prim.lambda.general] paragraph 3 as follows:

~~In the *decl-specifier-seq* of the *lambda-declarator*, each *decl-specifier* shall be one of *mutable*, *constexpr*, or *consteval*.~~ **A *lambda-specifier-seq* shall contain at most one of each *lambda-specifier* and shall not contain both *constexpr* and *consteval*.** If the *lambda-declarator* contains an explicit object parameter (9.3.4.6 [dcl.fct]), then no ~~*decl*~~**lambda-specifier** in the ~~*decl*~~**lambda-specifier-seq** shall be *mutable*.

2511. cv-qualified bit-fields

Section: 11.4.10 [class.bit] **Status:** ready **Submitter:** Aaron Ballman **Date:** 2021-09-15

According to 11.4.10 [class.bit] paragraph 1,

A bit-field shall have integral or enumeration type

This apparently does not allow for cv-qualification in a bit-field type.

Notes from the December, 2021 meeting:

As of N4901, there is no longer an issue regarding the integral types; 6.8.2 [basic.fundamental] paragraph 11 says,

The character types, `bool`, the signed and unsigned integer types, and cv-qualified versions (6.8.4 [basic.type.qualifier]) thereof, are collectively termed *integral types*.

Proposed resolution (December, 2021):

A bit-field shall have integral or **(possibly cv-qualified)** enumeration type

